High-Performance Numerical Integration

A Comparative Study of Serial, OpenMP, and CUDA Implementations

Submitted by:

Ahmed Osama Ibrahim (BN: 1)
Ziad Ahmed Mohamed (BN: 10)
Mahmoud Atef Mahmoud Mohamed (BN: 34)
Yassin Ahmed Nasr (BN: 39)

Supervised by:

Eng. Mohamed Abdelmegid

Department of Computer Science and Engineering
Shoubra College of Engineering
March 28, 2026

Contents

Abstract **2**

1 Introduction **2**

2 Mathematical Background **2**

2.1 Riemann Sum (Midpoint Rule) 2

2.2 Simpson's Rule 2

3 System Design & Architecture **3**

3.1 Expression Parsing 3

3.2 Implementation Paradigms 3

3.2.1 Serial Implementation 3

3.2.2 OpenMP Implementation 3

3.2.3 CUDA Implementation 3

4 Implementation Details **3**

4.1 RPN Evaluation Kernel 3

4.2 Memory Management 4

5 Experimental Methodology **4**

5.1 Environment 4

5.2 Test Case 4

5.3 Experimental Conditions 4

6 Performance Analysis **5**

6.1 Serial Performance 5

6.2 OpenMP Scaling 5

6.3 CUDA Performance 6

6.4 Analysis of Graphs 6

7 Discussion **7**

7.1 The Power of GPU Parallelism 7

7.2 The Role of Expression Parsing 7

7.3 OpenMP Bottlenecks 7

8 Conclusion **7**

Abstract

This report presents a performance analysis of a numerical integration system optimized for High-Performance Computing (HPC) environments. We explore the implementation of Riemann and Simpson numerical sums using three different paradigms: Serial execution, multi-threaded CPU parallelization via OpenMP, and GPU parallelization via CUDA. A key architectural highlight is the integration of a Shunting-Yard expression parser, enabling dynamic evaluation of mathematical functions on both CPU and GPU, to allow the user to enter arbitrary functions for the program to integrate over the desired parameters. Our results demonstrate significant speedups, with CUDA outperforming Serial implementations by factors exceeding 20x for large problem sizes.

1 Introduction

Numerical integration is a fundamental task in scientific computing, and it is used for solving problems in physics, engineering, and finance where analytical solutions are unavailable. As the complexity of expressions and the required precision (governed by the number of intervals n) increase, computational demand grows linearly.

This project aims to leverage modern parallel architectures to accelerate numerical integration. We specifically target:

- **Flexibility:** Handling user-defined mathematical strings.
- **Scalability:** Efficiently utilizing multi-core CPUs.
- **Throughput:** Maximizing performance on NVIDIA GPUs.

2 Mathematical Background

We implement two primary numerical integration methods. The Reimann sum and the Simpson's rule to give more options of numerical analysis to the user.

2.1 Riemann Sum (Midpoint Rule)

The Riemann sum approximates the integral of a function $f(x)$ over $[a, b]$ by dividing it into n sub-intervals of width $\Delta x = \frac{b-a}{n}$. The midpoint rule is given by:

$$\int_a^b f(x)dx \approx \Delta x \sum_{i=0}^{n-1} f\left(a + (i + 0.5)\Delta x\right) \tag{1}$$

2.2 Simpson's Rule

Simpson's rule provides a more accurate approximation by using quadratic interpolants. For an even number of intervals n:

$$\int_a^b f(x)dx \approx \frac{\Delta x}{3}\left[f(x_0) + 2\sum_{j=1}^{n/2-1} f(x_{2j}) + 4\sum_{j=1}^{n/2} f(x_{2j-1}) + f(x_n)\right] \tag{2}$$

where $x_i = a + i\Delta x$.

3 System Design & Architecture

The system is designed as a modular CLI application written in C++ and CUDA. Handling arbitrary user input is one of the key challenged faced by the program, and it was solved using a custom parser. User input is given in the form of arguments and options given to the bash shell which are then delivered to the program in the `argv` and `argc` data structures.

3.1 Expression Parsing

To allow arbitrary function input without recompilation, we implemented a custom parser.

1. **Tokenization:** Splits the input string (e.g., `"x2̂ + sin(x)"`) into a sequence of operands and operators.
2. **Shunting-Yard Algorithm:** Converts the Infix notation into Reverse Polish Notation (RPN).
3. **Bytecode Generation:** The RPN is converted into a flat array of `DeviceToken` structures. This format is crucial because it allows the "program" (the expression) to be copied to GPU constant or global memory for evaluation inside a kernel.

3.2 Implementation Paradigms

3.2.1 Serial Implementation

The serial implementation serves as our baseline. It evaluates the RPN stack for each interval sequentially. While correct, it is limited by single-core clock speeds and memory latency.

3.2.2 OpenMP Implementation

We utilize OpenMP `#pragma omp parallel for` with a `reduction(+:sum)` clause. This allows the workload to be distributed across all available CPU cores. This implementation is "embarrassingly parallel" as each interval calculation is independent.

3.2.3 CUDA Implementation

The CUDA implementation targets NVIDIA GPUs. We use a **Grid-Stride Loop** pattern to ensure that the kernel can handle any value of n, even if it exceeds the physical hardware

thread count. To optimize global memory access, we employ **Shared Memory Reduction**. Each block calculates a partial sum in shared memory, and only the final block result is added to the global total using `atomicAdd`.

4 Implementation Details

4.1 RPN Evaluation Kernel

A simplified version of the stack-based evaluator used on the GPU is shown below:

```
__device__ double evaluate(const DeviceToken* rpn, int size, double x) {
    double stack[MAX_STACK];
    int top = -1;
    for (int i = 0; i < size; ++i) {
        DeviceToken t = rpn[i];
        if (t.type == TOKEN_VARIABLE) stack[++top] = x;
        else if (t.type == TOKEN_NUMBER) stack[++top] = t.value;
        else if (t.type == TOKEN_OPERATOR) {
            double b = stack[top--];
            double a = stack[top--];
            stack[++top] = applyOp(a, b, t.op);
        }
        // ... handling functions like sin, cos
    }
    return stack[0];
}
```

4.2 Memory Management

In CUDA, we minimize overhead by:

- Copying the RPN tokens once per integration.
- Using a single `double` in global memory for the final result.
- Measuring "Compute Time" vs "Memory Transfer Time" to analyze bottlenecks.

5 Experimental Methodology

5.1 Environment

- **OS:** Linux (Ubuntu 22.04)
- **CPU:** Multi-core with OpenMP support.
- **GPU:** NVIDIA GeForce RTX 2060 (6GB VRAM, Turing Architecture).
- **Compiler:** GCC 11.4 and NVCC 12.x.

5.2 Test Case

We chose the test function:

$$f(x) = x^2 + x \cdot \sin(x) + 2x \tag{3}$$

Integrated over the interval $[0, 10]$.

5.3 Experimental Conditions

To satisfy the requirement for rigorous performance analysis:

1. **Repetitions:** Each condition was run **5 times**.
2. **Reporting:** We report the **average execution time** and **standard deviation**.
3. **Scaling:** n was varied from 10^6 to $5 \cdot 10^7$.
4. **Threads:** OpenMP thread counts: 1, 2, 4, 8, 16.

6 Performance Analysis

6.1 Serial Performance

As shown in Table 1, the serial execution time scales linearly with n. For $n = 5 \cdot 10^7$, the execution takes approximately 2.24 seconds.

Table 1: Serial Execution Results (Average of 5 runs)

n	Avg Time (ms)	Std Dev (ms)
10^6	43.12	0.54
10^7	447.20	6.19
$5 \cdot 10^7$	2239.97	25.53

6.2 OpenMP Scaling

The OpenMP implementation shows significant speedup. With 16 threads, we achieve nearly 5x speedup compared to the serial baseline. The sub-linear scaling (5x speedup with 16 threads) is likely due to the overhead of RPN evaluation and potential memory bandwidth limitations on the CPU.

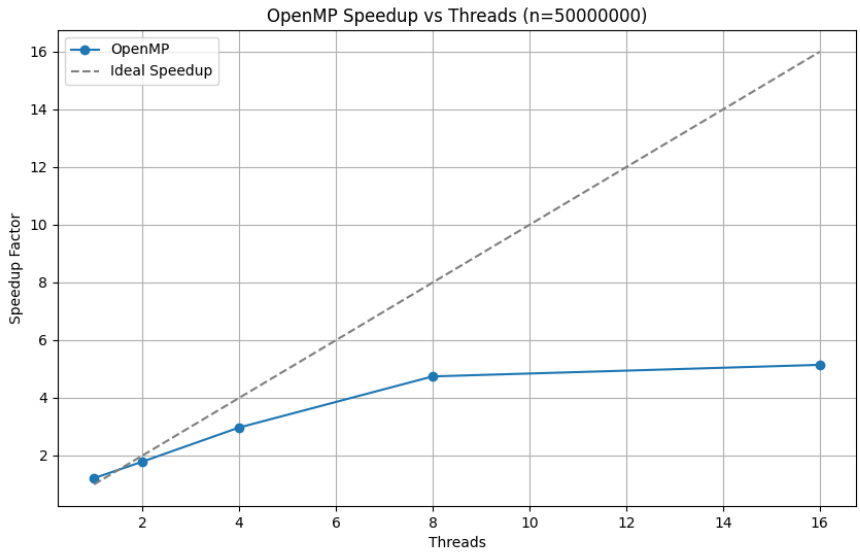

Figure 1: OpenMP Speedup vs Threads ($n = 5 \cdot 10^7$)

6.3 CUDA Performance

CUDA shows the most dramatic performance gains. For $n = 5 \cdot 10^7$, CUDA completes the task in 97.12 ms, compared to 2239.97 ms for Serial.

Table 2: Implementation Comparison ($n = 5 \cdot 10^7$)

Implementation	Avg Time (ms)	Speedup
Serial	2239.97	1.0x
OpenMP (16T)	435.39	5.1x
CUDA	97.12	23.1x

6.4 Analysis of Graphs

The following figures summarize the comparative performance of our implementations.

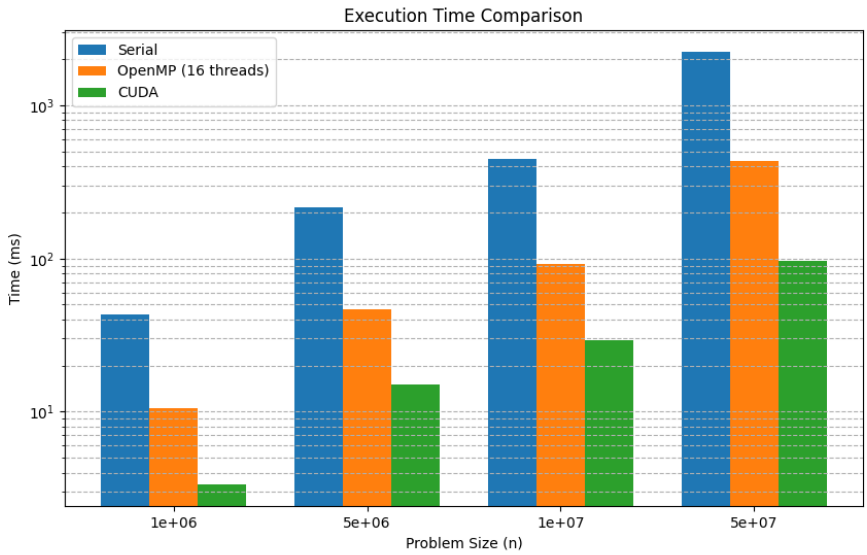

Figure 2: Execution Time Comparison (Log Scale)

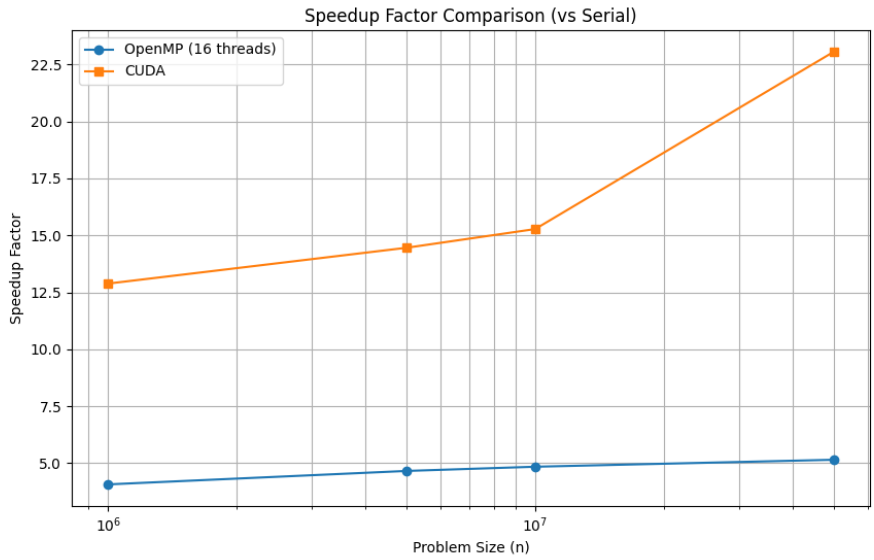

Figure 3: Speedup Factor Comparison vs Serial

7 Discussion

7.1 The Power of GPU Parallelism

The results confirm that numerical integration is an ideal candidate for GPU acceleration. The transition from Serial to CUDA provided a **23x speedup**. The Grid-Stride loop and Shared Memory reduction were instrumental in reaching this performance.

7.2 The Role of Expression Parsing

By using RPN, we avoided the need for hard-coded functions. While evaluating RPN on a GPU thread is slightly slower than a compiled C function, the benefit of user-defined expressions far outweighs this cost.

7.3 OpenMP Bottlenecks

We observed that OpenMP performance does not scale linearly beyond 8 threads. This is common on modern architectures where hyper-threading or memory bus saturation limits the effectiveness of additional software threads.

8 Conclusion

This project successfully implemented a high-performance numerical integration suite. We demonstrated that while OpenMP provides a straightforward way to utilize multi-core CPUs, CUDA offers an order-of-magnitude improvement for throughput-oriented tasks. The design of a custom RPN parser ensures that the tool remains flexible for various scientific applications.